

EXERCISE 4

Declare Variables and Process Data

ABAP Development Tools (ADT) • ABAP Cloud

CONFIDENTIALITY

The group number and access-related information are intentionally not included in this document. The class/package naming convention is shown generically where necessary.

1. Exercise Objective

Create an ABAP console application that stores two numbers, divides them, formats the result, and writes it to the ADT Console. Then control the result precision explicitly so decimal values are preserved.

2. Problem We Are Solving

A calculation can be mathematically correct but still produce an unwanted result when the ABAP data type does not support the required precision. This exercise demonstrates the difference between an inferred integer result and an explicitly defined decimal result.

3. Create the Global Class

Create the class in the ABAP package created in Exercise 2. Use the course naming pattern:

```
ZCL_<GROUP>_COMPUTE
```

Implement:

```
IF_OO_ADT_CLASSRUN
```

- Choose File → New → ABAP Class.
- Select the development package created in Exercise 2.
- Enter the COMPUTE class name using the course group-number naming convention.
- Add interface IF_OO_ADT_CLASSRUN.
- Choose Next and assign the existing exercise transport request.
- Choose Finish.

4. Task 1 — Declare Variables

Inside IF_OO_ADT_CLASSRUN~MAIN, declare two integer variables:

```
DATA number1 TYPE i.  
DATA number2 TYPE i.
```

Assign the input values:

```
number1 = -8.
number2 = 3.
```

What does TYPE i mean?

TYPE i is the ABAP integer type. It stores whole numbers and does not represent decimal places.

5. Task 2 — Calculate the Ratio

First, the exercise uses an inline declaration:

```
DATA(result) = number1 / number2.
```

ABAP infers the type of result from the expression. Because the input variables are integers, the result is inferred as an integer in this exercise.

6. Why Does -8 / 3 Produce -3?

```
-8 / 3 = -2.666666...
```

The inferred result is an integer, so decimal places cannot be retained. ABAP's integer conversion/rounding behavior produces -3. This is the important lesson: the data type affects the result.

Key lesson

Never choose or accept a data type blindly. For business calculations, think about precision, range, and the required output.

7. Build the Output Using a String Template

```
DATA(output) = |{ number1 } / { number2 } = { result }|.
```

A string template is text between `| |`. Expressions inside `{ }` are evaluated and inserted into the text.

```
|{ number1 } / { number2 } = { result }|
      ↓
    formatted text
```

8. Write to the Console

```
out->write( output ).
```

out is the output object provided to the class-run framework. `->` accesses an object method. `write( )` sends the supplied data to the ADT Console.

9. Task 3 — Activate and Test

- Press Ctrl + F3 to activate the class.
- Press F9 to execute the console application.
- Open the Console view if it is not visible.
- Check the result for -8 / 3.

Activate vs Execute

Ctrl + F3 activates the ABAP object. F9 executes the class-run application. They are different operations.

10. Task 4 — Control Result Precision

Replace the inline declaration of result with an explicit packed-decimal declaration:

```
DATA result TYPE p LENGTH 8 DECIMALS 2.
```

Then replace:

```
DATA(result) = number1 / number2.
```

with:

```
result = number1 / number2.
```

11. Understand TYPE p LENGTH 8 DECIMALS 2

Part	Meaning
TYPE p	Packed decimal numeric type.
LENGTH 8	Defines the packed-number storage length.
DECIMALS 2	Keeps two decimal places.

The calculation now keeps decimal precision:

```
-8 / 3 ≈ -2.67
```

12. Complete Model Code

```
CLASS zcl_<group>_compute DEFINITION
  PUBLIC
  FINAL
  CREATE PUBLIC .

  PUBLIC SECTION.
    INTERFACES if_oo_adt_classrun.

  PROTECTED SECTION.
  PRIVATE SECTION.
ENDCLASS.

CLASS zcl_<group>_compute IMPLEMENTATION.

  METHOD if_oo_adt_classrun~main.

    DATA number1 TYPE i.
    DATA number2 TYPE i.
    DATA result TYPE p LENGTH 8 DECIMALS 2.

    number1 = -8.
    number2 = 3.
```

```

    result = number1 / number2.

    DATA(output) = |{ number1 } / { number2 } = { result }|.

    out->write( output ).

ENDMETHOD.

ENDCLASS.

```

13. Code Explained

DATA number1 TYPE i.

Creates an integer variable named number1.

DATA number2 TYPE i.

Creates an integer variable named number2.

DATA result TYPE p LENGTH 8 DECIMALS 2.

Creates a decimal result with two decimal places.

number1 = -8.

Assigns -8 to number1.

number2 = 3.

Assigns 3 to number2.

result = number1 / number2.

Performs the division and stores the result.

DATA(output) = |...|.

Creates a string using an inline declaration and string template.

{ number1 } / { number2 } = { result }

Inserts the values into the output text.

out->write(output).

Displays the final output in the ABAP Console.

14. Technical Vocabulary

Term	Simple meaning
Variable	Named storage used to hold a value.
Data type	Defines what kind of value a variable stores and how it behaves.
Integer	Whole-number type with no decimal places.
Packed decimal	Numeric type suitable for decimal calculations.
Inline declaration	Declaring a variable where its value/expression is created.
Explicit declaration	Declaring a variable separately with a specified type.
String template	Text between that can contain expressions inside { }.

Precision	How accurately a numeric value can represent decimal detail.
Rounding	Changing a value according to the target numeric format.
Expression	Code that produces a value, such as number1 / number2.

15. Why This Matters in a Company

- Financial calculations depend on correct decimal precision.
- Prices, taxes, discounts, percentages, quantities, and exchange rates can be affected by data types.
- Incorrect inferred types can silently produce incorrect business results.
- Explicit data types make the developer's intention clear.
- Testing edge cases such as negative values and division helps expose data-type problems.

Real-world example

If a pricing calculation requires 12.75% but a developer accidentally stores the result as an integer, the business calculation can become wrong. The error is caused by the data type, not the arithmetic formula.

16. Interview Questions and Answers

1. What is a variable?

A named storage location used to hold a value during program execution.

2. What does TYPE i mean?

It defines an integer value without decimal places.

3. What is an inline declaration?

A declaration made directly where the value/expression is created, such as `DATA(result) = number1 / number2`.

4. Why can inline declaration be dangerous here?

Because ABAP infers the type and the inferred integer type loses decimal precision.

5. Why does -8 / 3 become -3 in the first version?

The result is an integer, so the fractional part cannot be retained and ABAP's integer conversion/rounding behavior produces -3.

6. How do we preserve two decimal places?

Declare result explicitly using a decimal-capable type such as `TYPE p ... DECIMALS 2`.

7. What is TYPE p?

ABAP packed decimal numeric type.

8. What is a string template?

A text expression between `| |` where `{ }` expressions are evaluated and inserted.

9. What does out->write() do?

It writes data to the ADT Console output.

10. Why is data-type selection important in SAP?

Because business calculations can become incorrect when range or decimal precision is unsuitable.

17. What Was Performed

- Created a COMPUTE global class in the development package.
- Implemented IF_OO_ADT_CLASSRUN.
- Declared two integer input variables.
- Assigned -8 and 3.
- Calculated the ratio.
- Observed the integer result caused by inferred typing.
- Created formatted output with a string template.
- Explicitly declared result as TYPE p with two decimal places.
- Activated and executed the class using Ctrl + F3 and F9.
- Verified the decimal result in the ABAP Console.

Core lesson

The main lesson is not the division itself. It is understanding how ABAP data types control calculation results and why explicit precision matters in real business applications.